

Indexing and reconstruction

Objective: map $I[B,C,H,W]$ to $F[B,C*H*W]$, then reconstruct I_{hat} . The batch index is preserved. Within each batch, width varies fastest, followed by height and channel. This gives a linear sequence suitable for tensor buffers and accelerator data transfers.

1. Forward mapping

```
F = allocate(B, C*H*W, same dtype as I)
for b in 0..B-1, c in 0..C-1:
    for h in 0..H-1, w in 0..W-1:
        k = (c*H + h)*W + w
        F[b,k] = I[b,c,h,w]
```

2. Reverse mapping

```
I_hat = allocate(B, C, H, W, same dtype as F)
for b in 0..B-1, k in 0..C*H*W-1:
    c = k // (H*W)
    r = k % (H*W)
    h = r // W; w = r % W
    I_hat[b,c,h,w] = F[b,k]
```

Euclidean division makes the mapping bijective: $k = c*(H*W) + h*W + w$, with $0 \leq h < H$ and $0 \leq w < W$. Both conversions take $O(B*C*H*W)$ time and allocate one output tensor. The reverse operation requires the original C , H and W as metadata.

3. Manual example: B=1, C=2, H=2, W=3

```
I[0,0,:,:] = [[0,1,2], [3,4,5]]
I[0,1,:,:] = [[6,7,8], [9,10,11]]
F[0,:] = [0,1,2,3,4,5,6,7,8,9,10,11]

(0,0,0,2) -> k=(0*2+0)*3+2=2 -> F[0,2]=2
(0,0,1,0) -> k=(0*2+1)*3+0=3 -> F[0,3]=3
(0,1,0,0) -> k=(1*2+0)*3+0=6 -> F[0,6]=6
(0,1,1,2) -> k=(1*2+1)*3+2=11 -> F[0,11]=11
```

Inverse example: $k=10$ gives $c=10//6=1$, $r=10\%6=4$, $h=4//3=1$, $w=4\%3=1$. Therefore $F[0,10]$ returns to $I_{\text{hat}}[0,1,1,1]=10$.

Experiments and measured errors

Each run checks every flattened element, reconstructed element and inverse index. The signed difference tensor $E = I - \hat{I}$ is computed explicitly and saved in errors/00.npy through errors/09.npy in table order. Metrics are calculated over all $B \times C \times H \times W$ elements.

$$\begin{aligned} E_{\max} &= \max \text{ over } b, c, h, w \text{ of } \text{abs}(E[b, c, h, w]) \\ \text{MAE} &= \text{sum over } b, c, h, w \text{ of } \text{abs}(E[b, c, h, w]) / (B \times C \times H \times W) \end{aligned}$$

4. Results (read directly from results.csv)

Input	B	C	H	W	E _{max}	MAE
Manual ordering	1	2	2	3	0.0	0.0
MNIST test samples 0-1	2	1	28	28	0.0	0.0
Synthetic RGB	2	3	32	32	0.0	0.0
Synthetic fmap C=8	2	8	13	17	0.0	0.0
Synthetic fmap C=16	2	16	13	17	0.0	0.0
Synthetic fmap C=32	2	32	13	17	0.0	0.0
Synthetic fmap C=64	2	64	13	17	0.0	0.0
Synthetic fmap C=128	2	128	13	17	0.0	0.0
Synthetic fmap C=256	2	256	13	17	0.0	0.0
Synthetic fmap C=500	2	500	13	17	0.0	0.0

10 / 10 experiments passed | 451,492 elements verified

MNIST uses the first two test images, unnormalized uint8 grayscale values. The three-channel input is synthetic RGB-shaped data, not CIFAR. All larger inputs are synthetic float32 feature maps. Synthetic samples use NumPy PCG64, seed = 20261005 + C, and a standard normal distribution.

All measured E_{max} and MAE values are 0.0; every saved element-wise difference is zero. No arithmetic is applied to values during either conversion, and dtype is preserved, so exact reconstruction is expected.

Validation and discussion

5. Verification

The 16 automated unit tests cover handwritten expected ordering, $B > 1$, non-square H/W, singleton dimensions, all nine requested channel counts and four numeric dtypes, noncontiguous inputs, index bijection, input preservation, invalid inputs and explicit-loop implementation. Deliberate matrix permutations and reconstruction corruption must be detected.

A separate nonzero-error test uses unsigned inputs with expected signed differences $[-255, 255, 2, -3]$, $E_{\max}=255$ and $MAE=128.75$. This checks the error calculator itself and guards against unsigned subtraction wraparound. Integer values are subtracted before conversion to floating-point metrics.

6. Observations and limitations

Zero error across $C=1$ through $C=500$ supports correct channel boundaries and preservation of the batch dimension. The independent ordering check is essential: a mutually consistent pair of wrong permutations could pass reconstruction alone. Quotient/remainder tests establish that every valid linear address has exactly one BCHW coordinate.

The implementation uses explicit Python loops and is intended to explain address generation, not benchmark accelerator throughput. It supports positive dimensions and finite real integer/floating-point NumPy tensors. Empty axes, NaN, infinity, complex and object values are rejected. Differences and aggregate metrics are stored as float64; exact integer subtraction precedes storage. No optional library layout-conversion reference is used.

7. Reproducibility and sources

Tested environment: Python 3.14.2, NumPy 2.4.2. Run `python -m unittest -v, python experiments.py, python build_report.py, and python verify_submission.py`. EVALUATOR.md provides setup, full provenance, an instruction-by-instruction audit and submission details.

Assignment source: `assgn3-tensor-flat.pdf`, pages 1-3 (provided by the user). MNIST source: CVDF's public mirror of the MNIST test images. The full archive is included in `data/` and verified by SHA-256; no network is needed to repeat the experiments.

Dataset reference: <https://github.com/cvdfoundation/mnist>

Results SHA-256: `cd5acec2ab9255e98642f3036381b3497a10c77e8e4f915b805053070f99e9ce`